Appendix: BPVS Estimation in R — Implementation Guide for `bpvs_model.R` and `bpvs_empirical.R`

Pathairat Pastpipatkul Htwe Ko

2025

Contents

1 Overview of the workflow

The Bayesian Panel Variable Selection (BPVS) implementation for this paper consists of two R scripts executed in sequence:

1. `bpvs_model.R` (Step 1) — defines the probability model, the prior structure, the fitting functions for the fixed-effects (BPVS-FE) and random-effects (BPVS-RE) variants, the posterior-inclusion-probability (PIP) computation, the ranking and selection rules, the model-fit statistics, and the Bayesian/frequentist Hausman comparisons. This file contains *functions only*; it must be sourced before anything is run.

2. `bpvs_empirical.R` (Step 2) — the empirical application driver. It loads and scales the data, builds the model formula, fits both variants by calling the Step 1 functions, saves the fitted models and result tables, and produces all figures.

A companion script, `bpvs_simulation.R`, uses the same Step 1 functions with synthetic data for the simulation study of the paper. The dependency structure is:

$$\texttt{bpvs\_model.R} \xrightarrow{\text{source()}} \texttt{bpvs\_simulation.R} \quad \text{and} \quad \texttt{bpvs\_empirical.R}$$

Both scripts run under R ≥ 4.0 with the packages `brms`, `tidyverse`, `posterior`, `bayesplot`, `loo`, `plm`, and `patchwork`/`ggrepel` installed. The sampling backend is Stan: `cmdstanr` if available, otherwise `rstan`.

2 Step 1: `bpvs_model.R`

2.1 What the file does

`bpvs_model.R` implements the full estimation pipeline as a set of reusable functions. Table 1 lists each function and its purpose.

Table 1: Functions defined in `bpvs_model.R`.

Function	Purpose
`fit_bpvs_fe()`	Fit BPVS-FE: unit dummy intercepts (unshrunk) + horseshoe on slopes.
`fit_bpvs_re()`	Fit BPVS-RE: random intercept $(1 \mid id)$ + horseshoe on slopes.
`fit_bpvs()`	Dispatcher: `effects = "fe"` or `"re"`.
`compute_pip()`	Estimate $\text{PIP}_j = \Pr(\lvert\beta_j\rvert > \varepsilon \mid data)$ from posterior draws.
`rank_variables()`	Rank predictors by PIP and label `Selected`/`Weak`/`Excluded`.
`extract_model_stats()`	Compute WAIC (with SE) and Bayesian R^2.
`print_bpvs_results()`	Pretty-print ranking, fit, and selection summary.
`bayesian_hausman()`	Posterior of $\Delta_j = \beta_j^{FE} - \beta_j^{RE}$ per variable.
`hausman_plm()`	Frequentist Hausman test via `plm`.
`compare_bpvs()`	Combine FE/RE rankings, Bayesian and frequentist Hausman.
`bpvs_pipeline()`	One-call driver: fit, compute, rank, print for FE and/or RE.

2.2 Key implementation choices

(i) FE via a nonlinear formula. BPVS-FE is specified as two additive blocks inside one `brms` formula,

$$y \sim \alpha + \eta, \qquad \alpha \sim 0 + id, \qquad \eta \sim 0 + x_1 + \cdots + x_p,$$

so that the unit dummies (block α) receive a wide `normal(0,10)` prior while the horseshoe (block η) is applied *only* to the slopes. This is the crucial device that keeps shrinkage away from the heterogeneity correction.

(ii) Regularized horseshoe. The prior

$$\beta_j \mid \lambda_j, \tau, c \;\sim\; \mathcal{N}\!\left(0,\; \frac{\tau^2 c^2 \lambda_j^2}{c^2 + \tau^2 \lambda_j^2}\right), \qquad \lambda_j \sim \mathcal{C}^+(0,1)$$

is set in `brms` by `set_prior("horseshoe(df = 1, scale_global = 2, autoscale = TRUE)", class = "b")`. With `autoscale = TRUE`, the global scale is $\tau_0 = \frac{p_0}{p - p_0} \cdot \frac{\sigma}{\sqrt{n}}$, so it adapts to the noise level and sample size.

(iii) PIP threshold. `compute_pip()` defines the effective-zero radius as $\varepsilon = 0.01 \times \mathrm{sd}(y)$ and estimates PIP_j as the posterior frequency of the event $\{|\beta_j^{(s)}| > \varepsilon\}$ over all MCMC draws s. Column prefixes differ by model: `b_eta_*` (FE) versus `b_*` (RE).

(iv) Sampling controls. Defaults are 4 chains, 4000 draws, 2000 warm-up, `adapt_delta` = 0.95, `max_treedepth` = 12, and `save_pars(all = TRUE)` so the latent horseshoe parameters remain in the saved object.

2.3 Full listing of `bpvs_model.R`

```
# ======================================================================
# bpvs_model.R --- Bayesian Panel Variable Selection (BPVS) Model
# Pathairat Pastpipatkul and Htwe Ko (2025)
# Revised: FE (unit dummies) and RE (random intercept) variants
# ======================================================================
#
# PROBABILITY MODEL:
#
#   Likelihood (Panel):
#     y_it ~ N(mu_it, sigma^2)
#     mu_it = alpha_i + sum_{j=1}^p beta_j * x_{itj}
#
#   BPVS-RE (random intercepts):
#     alpha_i ~ N(0, tau_unit^2)              # random effects (country effects)
#     fitted with:  y ~ x1 + ... + xp + (1 | id)
#
#   BPVS-FE (fixed effects, unit dummies):
#     alpha_i treated as free unit-specific intercepts
#     fitted with nonlinear brms formula:
#       y ~ alpha + eta
#       alpha ~ 0 + id                        # unshrunk unit dummies
#       eta  ~ 0 + x1 + ... + xp              # horseshoe only on slopes
#
#   Prior on slopes (Regularized Horseshoe --- continuous spike-and-slab):
#     beta_j ~ N(0, tau^2 * lambda_j^2)
#     lambda_j ~ Cauchy+(0, 1)                # local shrinkage
#     tau ~ Cauchy+(0, tau_0)                 # global shrinkage
#     tau_0 = p0/(p - p0) * sigma / sqrt(N)   # auto-scaled
#
#   Prior on variance components:
#     sigma ~ student_t(3, 0, 10)
#     tau_unit ~ exponential(1)
#
# IDENTIFICATION:
#   - FE: unit effects identified via dummy coefficients (proper wide prior)
#   - RE: alpha_i identified via proper prior variance tau_unit
#   - Horseshoe identified when p << N or sufficient MCMC iterations
#   - PIP_j = Pr(|beta_j| > eps | data), eps = 0.01 * sd(y)
```

```r
#
# OUTPUTS:
#   fit_bpvs_fe()        --- fits BPVS-FE via brms (unit dummies + horseshoe)
#   fit_bpvs_re()        --- fits BPVS-RE via brms (random intercept + horseshoe)
#   fit_bpvs()           --- dispatcher: effects = "fe" | "re"
#   compute_pip()        --- computes posterior inclusion probabilities
#   rank_variables()     --- ranks predictors by PIP with selection labels
#   extract_model_stats()--- WAIC and Bayesian R2
#   compare_bpvs()       --- FE vs RE comparison + Bayesian/Frequentist Hausman
#   bpvs_pipeline()      --- fit + compute + rank + print in one call
# ============================================================================

required_packages <- c("brms", "tidyverse", "posterior", "bayesplot", "loo", "plm")
for (pkg in required_packages) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    install.packages(pkg, repos = "https://cloud.r-project.org")
  }
  library(pkg, character.only = TRUE)
}

backend <- if (requireNamespace("cmdstanr", quietly = TRUE)) "cmdstanr" else "rstan"

# ----------------------------------------------------------------------------
# extract_slope_terms: pull predictor names from a linear formula RHS
# ----------------------------------------------------------------------------
extract_slope_terms <- function(formula) {
  t <- terms(formula)
  attr(t, "term.labels")
}

# ----------------------------------------------------------------------------
# fit_bpvs_fe: Fit BPVS with FIXED EFFECTS (unit dummies)
#   y ~ alpha + eta ;  alpha ~ 0 + id ;  eta ~ 0 + x1 + ... + xp
#   Horseshoe applied ONLY to slope block (eta); dummies unshrunk.
# ----------------------------------------------------------------------------
fit_bpvs_fe <- function(formula, data, id_var = "id",
                        chains = 4, iter = 4000, warmup = 2000, cores = 4,
                        seed = 123, adapt_delta = 0.95, max_treedepth = 12,
                        refresh = 500, ...) {

  if (!id_var %in% names(data)) {
    stop("id_var '", id_var, "' not found in data.")
  }
  data <- as.data.frame(data)
  data[[id_var]] <- factor(data[[id_var]])

  y_name <- all.vars(formula)[1]
  slopes <- extract_slope_terms(formula)
  if (length(slopes) == 0) stop("No predictors on RHS of formula.")

  nl_formula <- bf(
    as.formula(paste(y_name, "~ alpha + eta")),
    as.formula(paste("alpha ~ 0 +", id_var)),
    as.formula(paste("eta ~ 0 +", paste(slopes, collapse = " + "))),
```

```
    nl = TRUE
  )

  priors <- c(
    set_prior("horseshoe(df = 1, scale_global = 2, autoscale = TRUE)",
              nlpar = "eta", class = "b"),
    set_prior("normal(0, 10)", nlpar = "alpha", class = "b")
  )

  fit <- brm(
    formula    = nl_formula,
    data       = data,
    family     = gaussian(),
    prior      = priors,
    chains     = chains,
    iter       = iter,
    warmup     = warmup,
    cores      = cores,
    seed       = seed,
    backend    = backend,
    refresh    = refresh,
    save_pars  = save_pars(all = TRUE),
    control    = list(adapt_delta = adapt_delta, max_treedepth = max_treedepth),
    ...
  )
  attr(fit, "bpvs_effects") <- "fe"
  attr(fit, "bpvs_slopes")  <- slopes
  attr(fit, "bpvs_id_var")  <- id_var
  return(fit)
}

# ---------------------------------------------------------------------
# fit_bpvs_re: Fit BPVS with RANDOM EFFECTS (random intercepts)
# ---------------------------------------------------------------------
fit_bpvs_re <- function(formula, data, id_var = "id",
                        chains = 4, iter = 4000, warmup = 2000, cores = 4,
                        seed = 123, adapt_delta = 0.95, max_treedepth = 12,
                        refresh = 500, ...) {

  if (!id_var %in% names(data)) {
    stop("id_var '", id_var, "' not found in data.")
  }
  data <- as.data.frame(data)
  data[[id_var]] <- factor(data[[id_var]])

  y_name <- all.vars(formula)[1]
  slopes <- extract_slope_terms(formula)
  re_formula <- as.formula(paste(
    y_name, "~", paste(slopes, collapse = " + "), "+ (1 |", id_var, ")"
  ))

  fit <- brm(
    formula    = re_formula,
    data       = data,
```

```
    family     = gaussian(),
    prior      = c(set_prior("horseshoe(df = 1, scale_global = 2, autoscale = TRUE)",
                             class = "b")),
    chains     = chains,
    iter       = iter,
    warmup     = warmup,
    cores      = cores,
    seed       = seed,
    backend    = backend,
    refresh    = refresh,
    save_pars  = save_pars(all = TRUE),
    control    = list(adapt_delta = adapt_delta, max_treedepth = max_treedepth),
    ...
  )
  attr(fit, "bpvs_effects") <- "re"
  attr(fit, "bpvs_slopes")  <- slopes
  attr(fit, "bpvs_id_var")  <- id_var
  return(fit)
}

# ------------------------------------------------------------------------
# fit_bpvs: dispatcher
# ------------------------------------------------------------------------
fit_bpvs <- function(formula, data, effects = c("re", "fe"), id_var = "id", ...) {
  effects <- match.arg(effects)
  if (effects == "fe") {
    fit_bpvs_fe(formula, data, id_var = id_var, ...)
  } else {
    fit_bpvs_re(formula, data, id_var = id_var, ...)
  }
}

# ------------------------------------------------------------------------
# compute_pip: Compute Posterior Inclusion Probabilities
#   PIP_j = Pr(|beta_j| > eps | data), eps = effect_threshold * sd(y)
#   FE: slope block columns are b_eta_<var>
#   RE: slope columns are b_<var>
# ------------------------------------------------------------------------
compute_pip <- function(fit, data, outcome_var = "Income",
                        effect_threshold = 0.01) {
  post <- as_draws_df(fit)
  eps <- effect_threshold * sd(data[[outcome_var]], na.rm = TRUE)

  effects <- attr(fit, "bpvs_effects") %||% "re"
  pattern <- if (effects == "fe") "^b_eta_" else "^b_"

  b_names <- grep(pattern, names(post), value = TRUE)
  # RE: drop intercept; FE: eta block has no intercept
  b_names <- setdiff(b_names, "b_Intercept")

  pip <- sapply(b_names, function(col) {
    mean(abs(post[[col]]) > eps, na.rm = TRUE)
  })
  med   <- sapply(b_names, function(col) median(post[[col]], na.rm = TRUE))
```

```r
  q025  <- sapply(b_names, function(col) quantile(post[[col]], 0.025, na.rm = TRUE))
  q975  <- sapply(b_names, function(col) quantile(post[[col]], 0.975, na.rm = TRUE))

  # Clean variable names: strip "b_eta_" / "b_" prefix
  var_names <- gsub("^b_eta_|^b_", "", b_names)

  data.frame(
    Variable   = var_names,
    Estimate   = round(med, 6),
    CI_lower   = round(q025, 6),
    CI_upper   = round(q975, 6),
    PIP        = round(pip, 6),
    stringsAsFactors = FALSE
  )
}

# ------------------------------------------------------------------------
# rank_variables: Rank predictors by PIP with selection labels
# ------------------------------------------------------------------------
rank_variables <- function(pip_df) {
  pip_df %>%
    arrange(desc(PIP)) %>%
    mutate(
      Rank      = 1:n(),
      Selection = case_when(
        PIP > 0.5  ~ "Selected",
        PIP > 0.1  ~ "Weak",
        TRUE       ~ "Excluded"
      )
    )
}

# ------------------------------------------------------------------------
# extract_model_stats: Extract WAIC and Bayesian R2
# ------------------------------------------------------------------------
extract_model_stats <- function(fit) {
  w <- tryCatch(waic(fit), error = function(e) NULL)
  r2 <- tryCatch(bayes_R2(fit)[1], error = function(e) NA)
  if (!is.null(w)) {
    waic_val <- w$estimates["waic", "Estimate"]
    waic_se  <- w$estimates["waic", "SE"]
  } else {
    waic_val <- NA; waic_se <- NA
  }
  list(
    WAIC       = round(waic_val, 2),
    WAIC_SE    = round(waic_se, 2),
    Bayes_R2   = round(r2, 4)
  )
}

# ------------------------------------------------------------------------
# print_bpvs_results: Pretty-print BPVS output
# ------------------------------------------------------------------------
```

```r
print_bpvs_results <- function(ranking, stats, title = "BPVS") {
  cat("\n============================================================\n")
  cat(paste(title, "--- BAYESIAN PANEL VARIABLE SELECTION RESULTS\n"))
  cat("============================================================\n")

  cat("\n--- Variable Ranking (by Posterior Inclusion Probability) ---\n")
  print(ranking %>% select(Rank, Variable, Estimate, PIP, Selection),
        row.names = FALSE)

  cat(sprintf("\n--- Model Fit ---\n"))
  cat(sprintf("  WAIC:       %.2f (se = %.2f)\n", stats$WAIC, stats$WAIC_SE))
  cat(sprintf("  Bayes R2:   %.4f\n", stats$Bayes_R2))

  selected <- ranking$Variable[ranking$Selection == "Selected"]
  weak     <- ranking$Variable[ranking$Selection == "Weak"]

  cat("\n--- Variable Selection Summary ---\n")
  if (length(selected) > 0) {
    cat("  Selected (PIP > 0.50):", paste(selected, collapse = ", "), "\n")
  } else {
    cat("  Selected (PIP > 0.50): (none)\n")
  }
  if (length(weak) > 0) {
    cat("  Weak     (PIP > 0.10):", paste(weak, collapse = ", "), "\n")
  }
  excluded <- ranking$Variable[ranking$Selection == "Excluded"]
  if (length(excluded) > 0) {
    cat("  Excluded (PIP <= 0.10):", paste(excluded, collapse = ", "), "\n")
  }
  cat("============================================================\n")
}

# ------------------------------------------------------------------------
# bayesian_hausman: compare FE vs RE slope posteriors
#   For each variable, posterior of (beta_FE - beta_RE);
#   if the 95% interval excludes 0 -> RE inconsistent for that variable.
# ------------------------------------------------------------------------
bayesian_hausman <- function(fit_fe, fit_re, data, outcome_var = "Income",
                             effect_threshold = 0.01) {
  slopes <- attr(fit_fe, "bpvs_slopes") %||% attr(fit_re, "bpvs_slopes")

  post_fe <- as_draws_df(fit_fe)
  post_re <- as_draws_df(fit_re)

  delta_list <- lapply(slopes, function(v) {
    fe_col <- paste0("b_eta_", v)
    re_col <- paste0("b_", v)
    delta <- post_fe[[fe_col]] - post_re[[re_col]]
    data.frame(
      Variable = v,
      Delta    = delta
    )
  })
  delta_df <- do.call(rbind, delta_list)
```

```r

  delta_df %>%
    group_by(Variable) %>%
    summarise(
      Delta_median  = median(Delta),
      Delta_q025    = quantile(Delta, 0.025),
      Delta_q975    = quantile(Delta, 0.975),
      Pr_delta_not0 = mean(abs(Delta) > effect_threshold * sd(data[[outcome_var]], na.rm = TRUE)),
      .groups = "drop"
    ) %>%
    mutate(
      RE_Consistent = ifelse(Delta_q025 > 0 | Delta_q975 < 0, "No (RE biased)", "Yes")
    )
}

# ------------------------------------------------------------------------
# hausman_plm: Frequentist Hausman test via plm (auxiliary)
# ------------------------------------------------------------------------
hausman_plm <- function(formula, data, id_var = "id", time_var = "year") {
  if (!requireNamespace("plm", quietly = TRUE)) return(NULL)
  pdf <- tryCatch(
    plm::pdata.frame(as.data.frame(data), index = c(id_var, time_var)),
    error = function(e) NULL
  )
  if (is.null(pdf)) return(NULL)
  fe <- tryCatch(plm::plm(formula, data = pdf, model = "within"),
                 error = function(e) NULL)
  re <- tryCatch(plm::plm(formula, data = pdf, model = "random"),
                 error = function(e) NULL)
  if (is.null(fe) || is.null(re)) return(NULL)
  h <- tryCatch(plm::phtest(fe, re), error = function(e) NULL)
  if (is.null(h)) return(NULL)
  list(
    statistic = unname(h$statistic),
    p.value   = h$p.value
  )
}

# ------------------------------------------------------------------------
# compare_bpvs: FE vs RE comparison table + Hausman checks
# ------------------------------------------------------------------------
compare_bpvs <- function(fit_fe, fit_re, data, outcome_var = "Income",
                         formula = NULL, id_var = "id", time_var = "year",
                         effect_threshold = 0.01) {

  fe_pip <- compute_pip(fit_fe, data, outcome_var, effect_threshold)
  re_pip <- compute_pip(fit_re, data, outcome_var, effect_threshold)

  fe_rank <- rank_variables(fe_pip) %>% select(Variable, FE_Estimate = Estimate, FE_PIP = PIP,
                                               FE_Selection = Selection)
  re_rank <- rank_variables(re_pip) %>% select(Variable, RE_Estimate = Estimate, RE_PIP = PIP,
```

```
                                                RE_Selection = Selection)

  comparison <- merge(fe_rank, re_rank, by = "Variable", all = TRUE) %>%
    arrange(desc(FE_PIP))

  stats_fe <- extract_model_stats(fit_fe)
  stats_re <- extract_model_stats(fit_re)

  bh <- bayesian_hausman(fit_fe, fit_re, data, outcome_var, effect_threshold)
  comparison <- merge(comparison, bh, by = "Variable", all.x = TRUE)

  haus <- hausman_plm(formula, data, id_var, time_var)

  list(
    comparison = comparison,
    stats_fe   = stats_fe,
    stats_re   = stats_re,
    bayesian_hausman = bh,
    hausman_plm = haus
  )
}

# ------------------------------------------------------------------------
# print_compare: print FE vs RE comparison
# ------------------------------------------------------------------------
print_compare <- function(cmp) {
  cat("\n============================================================\n")
  cat("BPVS --- FE vs RE COMPARISON\n")
  cat("============================================================\n")

  cat("\n--- Model Fit ---\n")
  cat(sprintf("  FE: WAIC = %.2f (se %.2f), Bayes R2 = %.4f\n",
              cmp$stats_fe$WAIC, cmp$stats_fe$WAIC_SE, cmp$stats_fe$Bayes_R2))
  cat(sprintf("  RE: WAIC = %.2f (se %.2f), Bayes R2 = %.4f\n",
              cmp$stats_re$WAIC, cmp$stats_re$WAIC_SE, cmp$stats_re$Bayes_R2))

  cat("\n--- FE vs RE: Estimates and PIPs ---\n")
  print(cmp$comparison %>% select(Variable, FE_Estimate, RE_Estimate,
                                  FE_PIP, RE_PIP, FE_Selection, RE_Selection),
        row.names = FALSE)

  cat("\n--- Bayesian Hausman (posterior of beta_FE - beta_RE) ---\n")
  print(cmp$bayesian_hausman %>% select(Variable, Delta_median, Delta_q025,
                                        Delta_q975, Pr_delta_not0, RE_Consistent),
        row.names = FALSE)
  n_biased <- sum(cmp$bayesian_hausman$RE_Consistent == "No (RE biased)")
  if (n_biased > 0) {
    cat(sprintf("  %d variable(s) show FE/RE disagreement -> RE potentially biased there.
    \n", n_biased))
  } else {
    cat("  No FE/RE disagreement -> RE consistent with FE.\n")
  }

  if (!is.null(cmp$hausman_plm)) {
```

```
    cat("\n--- Frequentist Hausman (plm, auxiliary) ---\n")
    cat(sprintf("  Statistic = %.3f, p-value = %.4f\n",
                cmp$hausman_plm$statistic, cmp$hausman_plm$p.value))
    if (cmp$hausman_plm$p.value < 0.05) {
      cat("  p < 0.05 -> RE inconsistent; prefer BPVS-FE.\n")
    } else {
      cat("  p >= 0.05 -> no evidence against RE; BPVS-RE acceptable.\n")
    }
  }
  cat("================================================================\n")
}

# ------------------------------------------------------------------------
# bpvs_pipeline: fit (FE and/or RE) + compute + rank + print
# ------------------------------------------------------------------------
bpvs_pipeline <- function(formula, data, effects = c("fe", "re"),
                          outcome_var = "Income", id_var = "id",
                          time_var = "year", chains = 4, iter = 4000,
                          warmup = 2000, cores = 4, seed = 123,
                          adapt_delta = 0.95, max_treedepth = 12, ...) {

  effects <- match.arg(effects, several.ok = TRUE)

  results <- list()

  if ("fe" %in% effects) {
    cat("\n--- Fitting BPVS-FE (fixed effects, unit dummies) ---\n")
    fit_fe <- fit_bpvs_fe(formula, data, id_var = id_var, chains = chains,
                          iter = iter, warmup = warmup, cores = cores,
                          seed = seed, adapt_delta = adapt_delta,
                          max_treedepth = max_treedepth, ...)
    fe_pip  <- compute_pip(fit_fe, data, outcome_var)
    fe_rank <- rank_variables(fe_pip)
    fe_stat <- extract_model_stats(fit_fe)
    print_bpvs_results(fe_rank, fe_stat, title = "BPVS-FE")
    results$fit_fe   <- fit_fe
    results$rank_fe  <- fe_rank
    results$pip_fe   <- fe_pip
    results$stats_fe <- fe_stat
  }

  if ("re" %in% effects) {
    cat("\n--- Fitting BPVS-RE (random intercepts) ---\n")
    fit_re <- fit_bpvs_re(formula, data, id_var = id_var, chains = chains,
                          iter = iter, warmup = warmup, cores = cores,
                          seed = seed, adapt_delta = adapt_delta,
                          max_treedepth = max_treedepth, ...)
    re_pip  <- compute_pip(fit_re, data, outcome_var)
    re_rank <- rank_variables(re_pip)
    re_stat <- extract_model_stats(fit_re)
    print_bpvs_results(re_rank, re_stat, title = "BPVS-RE")
    results$fit_re   <- fit_re
    results$rank_re  <- re_rank
    results$pip_re   <- re_pip
```

```
    results$stats_re <- re_stat
  }

  if (all(c("fe", "re") %in% effects)) {
    cmp <- compare_bpvs(results$fit_fe, results$fit_re, data, outcome_var,
                        formula = formula, id_var = id_var, time_var = time_var)
    print_compare(cmp)
    results$comparison <- cmp
  }

  invisible(results)
}

`%||%` <- function(a, b) if (is.null(a)) b else a
```

3 Step 2: bpvs_empirical.R

3.1 What the file does

`bpvs_empirical.R` is the application driver for the growth-determinants study. Its execution flow is:

1. **Setup** — source `bpvs_model.R`; load `patchwork` and `ggrepel`; fix the seed (`seed = 2025123`).
2. **Data** — read `developing.csv` (25 countries $\times$ 24 years, $n = 600$); drop the country-name column; keep `year` and `id` as panel keys; identify the $p = 10$ numeric predictors (`IE, IH, Debt, FDI, INF, POP, TOP, UR, EXR, GR`).
3. **Standardization** — every predictor is centered and scaled to unit variance. This makes the horseshoe auto-scaling $\tau_0 \propto \sigma/\sqrt{n}$ well calibrated and makes the slopes comparable across variables.
4. **Formula** — build `Growth  IE + IH + Debt + FDI + INF + POP + TOP + UR + EXR + GR`.
5. **Estimation** — call `bpvs_pipeline()` with `effects = c("fe","re")`, 4 chains, 4000 draws, 2000 warm-up, `adapt_delta = 0.95`, `max_treedepth = 12`, seed 789.
6. **Saving** — write the fitted models (`bpvs_fit_fe.rds`, `bpvs_fit_re.rds`) and the result tables (`bpvs_ranking_fe.csv`, `bpvs_ranking_re.csv`, `bpvs_comparison.csv`, `bpvs_bayes_hausman.`
   `bpvs_hausman_plm.csv`) into `bpvs_emp_results/`.
7. **Figures** — inclusion-probability bars, coefficient plots with 95% intervals, a FE-vs-RE PIP scatter, and trace plots for convergence.

3.2 Outputs produced

Table 2: Outputs written to `bpvs_emp_results/`.

File	Contents
`bpvs_fit_fe.rds` / `bpvs_fit_re.rds`	Fitted `brmsfit` objects (FE and RE).
`bpvs_ranking_fe.csv` / `bpvs_ranking_re.csv`	Variable, median, 95% CI, PIP, rank, decision.
`bpvs_comparison.csv`	FE vs. RE estimates and PIPs plus Bayesian Hausman columr
`bpvs_bayes_hausman.csv`	Δ_j posterior summaries and $\Pr(\lvert\Delta_j\rvert > \varepsilon)$.
`bpvs_hausman_plm.csv`	Frequentist Hausman statistic and p-value.
`inclusion_probabilities_*.png`	PIP bar charts (FE, RE).
`coefficient_estimates_*.png`	Posterior median and 95% CI plots (FE, RE).
`pip_FE_vs_RE.png`	FE vs. RE PIP scatter.
`trace_plots_*.png`	Trace plots of slope parameters (FE, RE).

The console output consists of the variable rankings, the WAIC and Bayes R^2 of both models, the FE-vs-RE comparison table, the Bayesian Hausman table, and the frequentist Hausman test result (statistic $= 14.60$, $p = 0.148$).

3.3 Full listing of `bpvs_empirical.R`

```
# ============================================================================
# bpvs_empirical.R --- BPVS Real Data Application (FE and RE)
# Pathairat Pastpipatkul and Htwe Ko (2025)
# ============================================================================
#
# USAGE:
#   source("bpvs_model.R")
#   source("bpvs_empirical.R")
#
# OUTPUTS:
#   Console --- FE & RE rankings, PIPs, selection, comparison + Hausman
#   bpvs_emp_results/ --- PNG figures + saved models (.rds)
# ============================================================================

cat("============================================================\n")
cat("BPVS --- EMPIRICAL APPLICATION (FE and RE)\n")
cat("============================================================\n")
cat("\nSources: bpvs_model.R\n\n")

source("bpvs_model.R")
library(patchwork)
library(ggrepel)

set.seed(2025123)

# ----------------------------------------------------------------------
# 1. Load and prepare data
# ----------------------------------------------------------------------
```

```r

# --- EDIT THIS PATH to point to your CSV file ---
data_path <- "C:/Users/User/OneDrive/Desktop/developing.csv"

# --- EDIT THIS if your outcome column has a different name ---
outcome_var <- "Growth"
time_var    <- "year"
id_var      <- "id"

if (!file.exists(data_path)) {
  stop("Data file not found: ", data_path, "\n",
       "Please update 'data_path' in bpvs_empirical.R to point to your CSV.")
}

data_raw <- read.csv(data_path)
cat(sprintf("Loaded: %s\n", data_path))
cat(sprintf("Dimensions: %d rows x %d columns\n", nrow(data_raw), ncol(data_raw)))

# Drop character identifiers, ensure panel structure
data <- data_raw %>%
  dplyr::select(-any_of("Country")) %>%
  mutate(!!sym(id_var) := factor(.data[[id_var]]))

cat(sprintf("Panel: %d countries, %d time periods, %d total observations\n",
            n_distinct(data[[id_var]]), n_distinct(data[[time_var]]), nrow(data)))

if (!outcome_var %in% names(data)) {
  stop("Outcome variable '", outcome_var, "' not found. Adjust 'outcome_var'.")
}

# Identify numeric predictors (exclude id, year, outcome)
predictors <- setdiff(names(data), c(id_var, time_var, outcome_var))
cat("Predictors:", paste(predictors, collapse = ", "), "\n")

# Scale predictors for horseshoe prior
data_scaled <- data
for (v in predictors) {
  m <- mean(data_scaled[[v]], na.rm = TRUE)
  s <- sd(data_scaled[[v]], na.rm = TRUE)
  data_scaled[[v]] <- (data_scaled[[v]] - m) / s
}

# ------------------------------------------------------------------
# 2. Build formula
# ------------------------------------------------------------------
formula <- as.formula(paste(outcome_var, "~", paste(predictors, collapse = " + ")))
cat("Model formula:\n")
print(formula)

# ------------------------------------------------------------------
# 3. Fit BPVS-FE and BPVS-RE
# ------------------------------------------------------------------
cat("\nFitting BPVS-FE and BPVS-RE (horseshoe prior)...\n")

```

```r
res <- bpvs_pipeline(
  formula       = formula,
  data          = data_scaled,
  effects       = c("fe", "re"),
  outcome_var   = outcome_var,
  id_var        = id_var,
  time_var      = time_var,
  chains        = 4,
  iter          = 4000,
  warmup        = 2000,
  cores         = 4,
  seed          = 789,
  adapt_delta   = 0.95,
  max_treedepth = 12,
  refresh       = 1000
)

fit_fe  <- res$fit_fe
fit_re  <- res$fit_re
rank_fe <- res$rank_fe
rank_re <- res$rank_re
cmp     <- res$comparison

# ------------------------------------------------------------------------
# 4. Save models and results
# ------------------------------------------------------------------------
output_dir <- "bpvs_emp_results"
if (!dir.exists(output_dir)) dir.create(output_dir)

saveRDS(fit_fe, file.path(output_dir, "bpvs_fit_fe.rds"))
saveRDS(fit_re, file.path(output_dir, "bpvs_fit_re.rds"))
write.csv(rank_fe, file.path(output_dir, "bpvs_ranking_fe.csv"), row.names = FALSE)
write.csv(rank_re, file.path(output_dir, "bpvs_ranking_re.csv"), row.names = FALSE)
write.csv(cmp$comparison, file.path(output_dir, "bpvs_comparison.csv"), row.names = FALSE
    )
write.csv(cmp$bayesian_hausman, file.path(output_dir, "bpvs_bayes_hausman.csv"),
    row.names = FALSE)
if (!is.null(cmp$hausman_plm)) {
  write.csv(as.data.frame(cmp$hausman_plm), file.path(output_dir, "bpvs_hausman_plm.csv")
    ,
            row.names = FALSE)
}

# ------------------------------------------------------------------------
# 5. Visualizations
# ------------------------------------------------------------------------

# 5a. Inclusion probability bar chart (FE vs RE)
make_inclusion_plot <- function(ranking, title, filename) {
  ranking$Variable <- factor(ranking$Variable, levels = rev(ranking$Variable))
  p <- ggplot(ranking, aes(x = Variable, y = PIP)) +
    geom_segment(aes(xend = Variable, yend = 0), color = "gray60", linewidth = 0.5) +
    geom_point(aes(color = Selection), size = 4) +
    geom_hline(yintercept = 0.5, linetype = "dashed", color = "red", alpha = 0.5) +
```

```
    geom_hline(yintercept = 0.1, linetype = "dotted", color = "orange", alpha = 0.5) +
    scale_color_manual(values = c("Selected" = "#1b9e77", "Weak" = "#d95f02",
                                  "Excluded" = "#7570b3")) +
    labs(title = title,
         subtitle = "Dashed red = 0.5 (Selected), dotted orange = 0.1 (Weak signal)",
         x = "Predictor", y = "PIP = Pr(|beta| > eps | data)",
         color = "Decision") +
    ylim(0, 1.05) + coord_flip() +
    theme_minimal() + theme(legend.position = "bottom")
  ggsave(file.path(output_dir, filename), p, width = 8, height = max(4, 0.5 * nrow(
    ranking)))
  p
}

p_incl_fe <- make_inclusion_plot(rank_fe, "BPVS-FE: Posterior Inclusion Probabilities",
                                 "inclusion_probabilities_FE.png")
p_incl_re <- make_inclusion_plot(rank_re, "BPVS-RE: Posterior Inclusion Probabilities",
                                 "inclusion_probabilities_RE.png")

# 5b. Coefficient plot with 95% CIs
make_coef_plot <- function(ranking, title, filename) {
  ranking$Variable <- factor(ranking$Variable, levels = ranking$Variable)
  p <- ggplot(ranking, aes(x = Estimate, y = Variable)) +
    geom_vline(xintercept = 0, linetype = "dashed", alpha = 0.3) +
    geom_pointrange(aes(xmin = CI_lower, xmax = CI_upper, color = Selection),
                    size = 0.9, fatten = 2.5) +
    scale_color_manual(values = c("Selected" = "#1b9e77", "Weak" = "#d95f02",
                                  "Excluded" = "#7570b3")) +
    labs(title = title,
         subtitle = "Horseshoe prior shrinks irrelevant coefficients toward zero",
         x = "Posterior Median (95% CI)", y = "Predictor", color = "Decision") +
    theme_minimal() + theme(legend.position = "bottom")
  ggsave(file.path(output_dir, filename), p, width = 8, height = max(4, 0.6 * nrow(
    ranking)))
  p
}

p_coef_fe <- make_coef_plot(rank_fe, "BPVS-FE: Coefficient Estimates (95% CI)",
                            "coefficient_estimates_FE.png")
p_coef_re <- make_coef_plot(rank_re, "BPVS-RE: Coefficient Estimates (95% CI)",
                            "coefficient_estimates_RE.png")

# 5c. PIP comparison FE vs RE
pip_compare <- merge(
  rank_fe %>% select(Variable, PIP_FE = PIP),
  rank_re %>% select(Variable, PIP_RE = PIP),
  by = "Variable", all = TRUE
)
p_pip_compare <- ggplot(pip_compare, aes(x = PIP_FE, y = PIP_RE, label = Variable)) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "gray50") +
  geom_point(size = 4, color = "steelblue", alpha = 0.8) +
  geom_text_repel(size = 3.5) +
  coord_fixed(xlim = c(0, 1), ylim = c(0, 1)) +
  labs(title = "BPVS: Posterior Inclusion Probabilities --- FE vs RE",
```

```
       x = "BPVS-FE PIP", y = "BPVS-RE PIP") +
  theme_minimal()
ggsave(file.path(output_dir, "pip_FE_vs_RE.png"), p_pip_compare, width = 6, height = 6)

# 5d. Trace plots for convergence (slopes)
beta_params_fe <- grep("^b_eta_", variables(fit_fe), value = TRUE)
if (length(beta_params_fe) > 12) beta_params_fe <- beta_params_fe[1:12]
p_trace_fe <- mcmc_trace(fit_fe, pars = beta_params_fe, facet_args = list(ncol = 3))
ggsave(file.path(output_dir, "trace_plots_FE.png"), p_trace_fe,
       width = 12, height = 3 * ceiling(length(beta_params_fe) / 3))

beta_params_re <- setdiff(grep("^b_", variables(fit_re), value = TRUE), "b_Intercept")
if (length(beta_params_re) > 12) beta_params_re <- beta_params_re[1:12]
p_trace_re <- mcmc_trace(fit_re, pars = beta_params_re, facet_args = list(ncol = 3))
ggsave(file.path(output_dir, "trace_plots_RE.png"), p_trace_re,
       width = 12, height = 3 * ceiling(length(beta_params_re) / 3))

# Display
print(p_incl_fe)
print(p_incl_re)
print(p_coef_fe)
print(p_coef_re)
print(p_pip_compare)

cat(sprintf("\nResults saved to: %s/\n", output_dir))
cat("============================================================\n")
cat("EMPIRICAL APPLICATION COMPLETE\n")
cat("============================================================\n")
```

4 Reproducing the results

To reproduce the empirical application:

1. Ensure the packages in Section 1 are installed and Stan is working.
2. Edit `data_path` in `bpvs_empirical.R` to point to `developing.csv`.
3. Run from the project directory:

```
source("bpvs_model.R")
source("bpvs_empirical.R")
```

The simulation study is reproduced analogously with `bpvs_simulation.R`. Because Stan sampling is stochastic, posterior summaries may differ slightly between runs, but the substantive conclusions (top predictors, FE–RE agreement, Hausman result) are stable under the fixed seeds reported above.